Gateway User Manual

Last updated: 2025-12-09

What It Is

- A resident process with a single Baileys/Telegram connection and control/event plane.
- Replaces the legacy `gateway` command. CLI entry point: `openclaw gateway`.
- Runs until stopped; exits with a non-zero exit code on fatal errors so supervisors can restart it.

How to Run (Local)

```
openclaw gateway --port 18789
# Get full debug/trace logs in stdio:
openclaw gateway --port 18789 --verbose
# If port is occupied, kill the listener then start:
openclaw gateway --force
# Development loop (auto-reload on TS changes):
pnpm gateway:watch
```

- Configuration hot-reload watches `~/.openclaw/openclaw.json` (or `OPENCLAW_CONFIG_PATH`).
  - Default mode: `gateway.reload.mode="hybrid"` (hot-apply safe changes, restart on critical changes).
  - Hot-reload uses in-process restart via **SIGUSR1** when needed.
  - Disable with `gateway.reload.mode="off"`.
- Bind WebSocket control plane to `127.0.0.1:<port>` (default 18789).
- The same port also serves HTTP (control UI, hooks, A2UI). Single-port multiplexing.
  - OpenAI Chat Completions (HTTP): `/v1/chat/completions`.
  - OpenResponses (HTTP): `/v1/responses`.
  - Tools Invoke (HTTP): `/tools/invoke`.
- Starts Canvas file server on `canvasHost.port` (default `18793`) by default, serving `http://<gateway-host>:18793/__openclaw__/canvas/` from `~/.openclaw/workspace/canvas`. Disable with `canvasHost.enabled=false` or `OPENCLAW_SKIP_CANVAS_HOST=1`.
- Outputs logs to stdout; use launchd/systemd to keep running and rotate logs.
- Pass `--verbose` when troubleshooting to mirror debug logs (handshake, request/response, events) from log file to stdio.
- `--force` uses `lsof` to find listeners on the selected port, sends SIGTERM, logs what it killed, then starts the Gateway (fails fast if `lsof` is missing).
- If you're running under a supervisor (launchd/systemd/macOS app subprocess mode), stop/restart typically sends **SIGTERM**; older versions may show this as `pnpm ELIFECYCLE` exit code **143** (SIGTERM), which is normal shutdown, not a crash.
- **SIGUSR1** triggers in-process restart when authorized (Gateway tool/config apply/update, or `commands.restart` enabled for manual restart).

- Gateway auth is required by default: set `gateway.auth.token` (or `OPENCLAW_GATEWAY_TOKEN`) or `gateway.auth.password`. Clients must send `connect.params.auth.token/password` unless using Tailscale Serve identity.
- The wizard now generates tokens by default, even on loopback.
- Port priority: `--port` > `OPENCLAW_GATEWAY_PORT` > `gateway.port` > default `18789`.

Remote Access

- Preferred: Tailscale/VPN; otherwise use SSH tunnels:

```
ssh -N -L 18789:127.0.0.1:18789 user@host
```

- Then clients connect through the tunnel to `ws://127.0.0.1:18789`.
- If a token is configured, clients must include it in `connect.params.auth.token` even through the tunnel.

Multiple Gateways (Same Host)

Usually not needed: one Gateway can serve multiple messaging channels and agents. Only use multiple Gateways when you need redundancy or strict isolation (e.g., rescue bot). Supported if you isolate state + config and use unique ports. Full guide: Multiple Gateways. Service names are profile-aware:

- macOS: `bot.molt.<profile>` (legacy `com.openclaw.*` may still exist)
- Linux: `openclaw-gateway-<profile>.service`
- Windows: `OpenClaw Gateway (<profile>)`

Installation metadata embedded in service config:

- `OPENCLAW_SERVICE_MARKER=openclaw`
- `OPENCLAW_SERVICE_KIND=gateway`
- `OPENCLAW_SERVICE_VERSION=<version>`

Rescue bot mode: Keep a second Gateway isolated with its own config, state directory, workspace, and base port offset. Full guide: Rescue Bot Guide.

Dev Profile (`--dev`)

Quick path: Run a fully isolated dev instance (config/state/workspace) without touching your main setup.

```
openclaw --dev setup
openclaw --dev gateway --allow-unconfigured
# Then target the dev instance:
openclaw --dev status
openclaw --dev health
```

Defaults (overridable via env/flags/config):

- `OPENCLAW_STATE_DIR=~/.openclaw-dev`
- `OPENCLAW_CONFIG_PATH=~/.openclaw-dev/openclaw.json`
- `OPENCLAW_GATEWAY_PORT=19001` (Gateway WS + HTTP)
- Browser control service port = `19003` (derived: `gateway.port+2`, loopback only)

- `canvasHost.port=19005` (derived: `gateway.port+4`)
- When you run `setup`/`onboard` under `--dev`, `agents.defaults.workspace` defaults to `~/.openclaw/workspace-dev`.

Derived ports (rule of thumb):

- Base port = `gateway.port` (or `OPENCLAW_GATEWAY_PORT` / `--port`)
- Browser control service port = base + 2 (loopback only)
- `canvasHost.port = base + 4` (or `OPENCLAW_CANVAS_HOST_PORT` / config override)
- Browser profile CDP ports auto-allocated from `browser.controlPort + 9 .. + 108` (persisted per profile).

Per-instance checklist:

- Unique `gateway.port`
- Unique `OPENCLAW_CONFIG_PATH`
- Unique `OPENCLAW_STATE_DIR`
- Unique `agents.defaults.workspace`
- Separate WhatsApp number (if using WA)

Install service by profile:

```
openclaw --profile main gateway install
openclaw --profile rescue gateway install
```

Example:

```
OPENCLAW_CONFIG_PATH=~/.openclaw/a.json OPENCLAW_STATE_DIR=~/.openclaw-a openclaw
gateway --port 19001
OPENCLAW_CONFIG_PATH=~/.openclaw/b.json OPENCLAW_STATE_DIR=~/.openclaw-b openclaw
gateway --port 19002
```

Protocol (Operations Perspective)

- Full docs: Gateway Protocol and Bridge Protocol (Legacy).
- First frame clients must send: `req {type:"req", id, method:"connect", params: {minProtocol,maxProtocol,client: {id,displayName?,version,platform,deviceFamily?,modelIdentifier?,mode,instanceId?}, caps, auth?, locale?, userAgent? } }`.
- Gateway replies `res {type:"res", id, ok:true, payload:hello-ok }` (or `ok:false` with error, then closes).
- After handshake:
  - Request: `{type:"req", id, method, params}` → `{type:"res", id, ok, payload|error}`
  - Event: `{type:"event", event, payload, seq?, stateVersion?}`
- Structured presence entries: `{host, ip, version, platform?, deviceFamily?, modelIdentifier?, mode, lastInputSeconds?, ts, reason?, tags?[], instanceId? }` (for WS clients, `instanceId` comes from `connect.client.instanceId`).
- `agent` responses are two-phase: first `res` acknowledges `{runId,status:"accepted"}`, then a final `res` `{runId,status:"ok"|"error",summary}` when the run completes; streamed output arrives as `event:"agent"`.

Methods (Initial Set)

- `health` — Full health snapshot (same shape as `openclaw health --json`).
- `status` — Short summary.
- `system-presence` — Current presence list.
- `system-event` — Publish presence/system notes (structured).
- `send` — Send message through active channel.
- `agent` — Run an agent turn (events flow back on the same connection).
- `node.list` — List paired + currently connected nodes (includes `caps`, `deviceFamily`, `modelIdentifier`, `paired`, `connected`, and broadcast `commands`).
- `node.describe` — Describe a node (capabilities + supported `node.invoke` commands; works for paired nodes and currently connected unpaired nodes).
- `node.invoke` — Invoke a command on a node (e.g., `canvas.*`, `camera.*`).
- `node.pair.*` — Pairing lifecycle (`request`, `list`, `approve`, `reject`, `verify`).

See also: Presence for how presence is produced/deduplicated and why stable `client.instanceId` matters.

Events

- `agent` — Streamed tool/output events from agent runs (with seq markers).
- `presence` — Presence updates (incremental with stateVersion) pushed to all connected clients.
- `tick` — Periodic keepalive/no-op to confirm liveness.
- `shutdown` — Gateway is exiting; payload includes `reason` and optional `restartExpectedMs`. Clients should reconnect.

WebChat Integration

- WebChat is a native SwiftUI UI that communicates directly with the Gateway WebSocket for history, sending, abort, and events.
- Remote usage via the same SSH/Tailscale tunnel; if Gateway token is configured, clients include it during `connect`.
- macOS app uses a single WS connection (shared connection); it populates presence from the initial snapshot and listens to `presence` events to update the UI.

Types and Validation

- Server validates every inbound frame using AJV against JSON Schema emitted from protocol definitions.
- Clients (TS/Swift) consume generated types (TS directly; Swift via generator in the repo).
- Protocol definitions are the source of truth; regenerate schema/models with:
  - `pnpm protocol:gen`
  - `pnpm protocol:gen:swift`

Connection Snapshot

- `hello-ok` contains a `snapshot` with `presence`, `health`, `stateVersion`, and `uptimeMs`, plus `policy {maxPayload,maxBufferedBytes,tickIntervalMs}`, so clients can render immediately without additional requests.

- `health` / `system-presence` are still available for manual refresh but not required at connection time.

Error Codes (res.error shape)

- Errors use `{ code, message, details?, retryable?, retryAfterMs? }`.
- Standard codes:
  - `NOT_LINKED` — WhatsApp not authenticated.
  - `AGENT_TIMEOUT` — Agent did not respond within configured deadline.
  - `INVALID_REQUEST` — Schema/parameter validation failed.
  - `UNAVAILABLE` — Gateway is shutting down or dependency unavailable.

Keepalive Behavior

- `tick` events (or WS ping/pong) fire periodically so clients know the Gateway is alive even when there's no traffic.
- Send/agent acknowledgments are kept separate; don't overload tick for sends.

Replay / Gaps

- Events are not replayed. Clients detecting seq gaps should flush (`health` + `system-presence`) before continuing. WebChat and macOS clients now auto-flush on gaps.

Supervision (macOS Example)

- Use launchd to keep the service alive:
  - Program: path to `openclaw`
  - Arguments: `gateway`
  - KeepAlive: true
  - StandardOut/Err: file path or `syslog`
- launchd restarts on failure; fatal config errors should keep exiting so operators notice.
- LaunchAgents are per-user, require a logged-in session; for headless setups, use a custom LaunchDaemon (not bundled).
  - `openclaw gateway install` writes `~/Library/LaunchAgents/bot.molt.gateway.plist` (or `bot.molt.<profile>.plist`; legacy `com.openclaw.*` is cleaned up).
  - `openclaw doctor` audits LaunchAgent config and can update it to current defaults.

Gateway Service Management (CLI)

Use the Gateway CLI for install/start/stop/restart/status:

```
openclaw gateway status
openclaw gateway install
openclaw gateway stop
openclaw gateway restart
openclaw logs --follow
```

Notes:

- `gateway status` probes Gateway RPC via the service-resolved port/config by default (override with `--url`).
- `gateway status --deep` adds system-level scan (LaunchDaemons/system units).
- `gateway status --no-probe` skips RPC probe (useful on network failures).
- `gateway status --json` is stable for scripting.
- `gateway status` separates **supervisor runtime** (launchd/systemd running) from **RPC reachability** (WS connection + status RPC).
- `gateway status` prints config path + probe target to avoid "localhost vs LAN binding" confusion and config file mismatches.
- `gateway status` includes the last line of Gateway error when the service appears to be running but the port is closed.
- `logs` tails Gateway file logs via RPC (no manual `tail`/`grep`).
- The CLI warns if it detects other Gateway-like services, unless they are OpenClaw profile services. We still recommend

  one Gateway per machine

  ; use isolated config files/ports for redundancy or rescue bots. See

  Multiple Gateways

  .
  - Cleanup: `openclaw gateway uninstall` (current service) and `openclaw doctor` (legacy migration).
- `gateway install` is a no-op when already installed; use `openclaw gateway install --force` to reinstall (config file/env/path changes).

Bundled macOS app:

- OpenClaw.app can bundle the Node-based Gateway relay and install a per-user LaunchAgent tagged as `bot.molt.gateway` (or `bot.molt.<profile>`; legacy `com.openclaw.*` tags uninstall cleanly).
- To stop it cleanly, use `openclaw gateway stop` (or `launchctl bootout gui/$UID/bot.molt.gateway`).
- To restart, use `openclaw gateway restart` (or `launchctl kickstart -k gui/$UID/bot.molt.gateway`).
  - `launchctl` only works when the LaunchAgent is installed; otherwise run `openclaw gateway install` first.
  - When running a named profile, replace the tag with `bot.molt.<profile>`.

Supervision (systemd User Unit)

OpenClaw installs **systemd user services** by default on Linux/WSL2. We recommend user services for single-user machines (simpler env, per-user config). For multi-user or resident servers use **system services** (no lingering, shared supervision). `openclaw gateway install` writes the user unit. `openclaw doctor` audits the unit and can update it to match current recommended defaults. Create `~/.config/systemd/user/openclaw-gateway[-<profile>].service`:

```
[Unit]
Description=OpenClaw Gateway (profile: <profile>, v<version>)
After=network-online.target
```

```
Wants=network-online.target

[Service]
ExecStart=/usr/local/bin/openclaw gateway --port 18789
Restart=always
RestartSec=5
Environment=OPENCLAW_GATEWAY_TOKEN=
WorkingDirectory=/home/youruser

[Install]
WantedBy=default.target
```

Enable lingering (required so user services survive logout/idle):

```
sudo loginctl enable-linger youruser
```

The wizard runs this command on Linux/WSL2 (may prompt for sudo; writes to `/var/lib/systemd/linger`). Then enable the service:

```
systemctl --user enable --now openclaw-gateway[-<profile>].service
```

Alternative (system service) - For resident or multi-user servers, you can install systemd **system** units instead of user units (no lingering required). Create `/etc/systemd/system/openclaw-gateway[-<profile>].service` (copy the unit above, switch `WantedBy=multi-user.target`, set `User=` + `WorkingDirectory=`), then:

```
sudo systemctl daemon-reload
sudo systemctl enable --now openclaw-gateway[-<profile>].service
```

Windows (WSL2)

Windows installations should use **WSL2** and follow the Linux systemd section above.

Operations Checklist

- Liveness check: Open WS and send `req:connect` → expect `res` with `payload.type="hello-ok"` (with snapshot).
- Readiness check: Call `health` → expect `ok: true` and linked channel in `linkChannel` (when applicable).
- Debug: Subscribe to `tick` and `presence` events; ensure `status` shows linked/authenticated time; presence entries show Gateway host and connected clients.

Security Guarantees

- Default assumption: one Gateway per host; if you run multiple profiles, isolate ports/state and target the correct instance.
- Never falls back to direct Baileys connection; sends fail fast if Gateway is down.
- Non-connect first frame or malformed JSON is rejected and socket closed.
- Graceful shutdown: emits `shutdown` event before closing; clients must handle close + reconnect.

CLI Helpers

- `openclaw gateway health|status` — Request health/status via Gateway WS.
- `openclaw message send --target <num> --message "hi" [--media ...]` — Send via Gateway (idempotent for WhatsApp).
- `openclaw agent --message "hi" --to <num>` — Run agent turn (waits for final result by default).
- `openclaw gateway call <method> --params '{"k":"v"}'` — Raw method invoker for debugging.
- `openclaw gateway stop|restart` — Stop/restart supervised Gateway service (launchd/systemd).
- Gateway helper subcommands assume a running Gateway on `--url`; they no longer auto-spawn one.

Migration Guide

- Deprecate use of `openclaw gateway` and legacy TCP control port.
- Update clients to use WS protocol with mandatory connect and structured presence.

For more about Gateway, see: [Gateway User Manual - OpenClaw]